Small Model, Honest Numbers: A Distilled, Vocabulary-Pruned DeBERTa-v3-xsmall for RTE

[Student name] [Roll number] CS6315E Neural Networks and Deep Learning
Code and predictions: submitted with this report (see the README)

1 Approach

Model choice. Since accuracy is weighed against size, I compared three sizes of one pretrained family, DeBERTa-v3 [2, 3] (base 184M, small 142M, xsmall 71M), with identical code. v3-base is my reference baseline and the distillation teacher; v3-xsmall is the student.

Fine-tuning protocol. One config-driven training path for all models: AdamW, 10% linear warm-up then decay, layer-wise LR decay 0.9, separate head LR, max length 256, batch 16, early stopping on validation accuracy. With 2,490 training pairs results swing with the seed [7], so every configuration uses 5 seeds (mean ± sd). All decisions (early stopping, checkpoint and model selection, pruning level) use a 10% validation split of train that is *grouped by premise*, so near-duplicate pairs never straddle it. The dev set (277) is used only to report.

Distillation. Teacher soft labels are computed *out-of-fold*: five DeBERTa-v3-base models, each trained on four premise-grouped folds, predict the fifth (OOF accuracy 82.9%). The student minimises $\alpha\,\mathrm{CE}(y, s) + (1-\alpha)\,T^2\,\mathrm{KL}(\sigma(t/T)\,\|\,\sigma(s/T))$ [4] with α=0.5, T=2, plus two variants: *masked* KD (pure CE on rows where the teacher's argmax is wrong) and α=0.8, T=4.

Vocabulary pruning. 49.2M of xsmall's 70.8M parameters are the 128,100 × 384 word embedding, yet RTE uses about 19k tokens. After training, I keep only the embedding rows of: special tokens ∪ every training-set token ∪ the top-N tokens ranked by the tokenizer's own unigram log-probabilities [5]; all other ids map to `[UNK]` through a fixed lookup table [1]. On input made only of kept tokens the pruned model is *exactly* the original. N is chosen on train alone: the smallest N for which one half of train covers 98% of the other half's tokens (giving N=60,000, 62,245 rows kept). I also tried a rank-128 SVD factorisation of the embedding (38.1M parameters).

2 Results

The submission has **4.0× fewer parameters** than DeBERTa-v3-base (45.5M vs. 184.4M) and 2.1× fewer non-embedding parameters than v3-small. Against my v3-base baseline it is 7.2 points lower on dev for the selected checkpoints (85.6 vs. 78.3; McNemar p=0.012, 39 vs. 19 discordant pairs) and 6.5 points lower on 5-seed means. Pruning flipped 9 of 277 dev predictions (5 now right, 4 now wrong; p=1.0). A Colab rerun of seed 21 reproduced the checkpoint bit for bit; an Apple-MPS rerun did not (dev 74.7).

Plain distillation copies the teacher's bias; masking it out helps a little. KD at α=0.5 scored −0.9 points versus plain fine-tuning (paired t over seeds, p=0.27). It improved calibration (dev log-loss 0.74 vs. 0.99) and seed agreement (92% vs. 88%), but the teacher is biased: on train it predicts *entailment* for 57.8% of pairs against 50.2% gold, and 72% of its OOF errors are on not-entailment pairs. The KD student inherits it (61.5% entailment predictions on dev vs. 52.7% gold). Masked KD removes the inherited bias (55.8%, same as plain) and is the best method on

Model	Total params	Non-emb.	Dev acc. (5 seeds)	Val acc.
Majority / word overlap / TF-IDF + LR	–	–	52.7 / 57.4 / 57.0	–
Instructor reference baseline	[TBD]	[TBD]	[TBD]	–
DeBERTa-v3-base (my baseline, teacher)	184.4M	86.0M	84.8 ± 1.1	86.2
DeBERTa-v3-small	141.9M	43.5M	76.5 ± 1.4	75.1
DeBERTa-v3-xsmall, plain fine-tuning	70.8M	21.6M	77.7 ± 2.1	78.5
+ KD (α 0.5, T 2)	70.8M	21.6M	76.8 ± 1.6	78.4
+ masked KD (α 0.5, T 2)	70.8M	21.6M	78.3 ± 1.4	79.2
+ KD (α 0.8, T 4)	70.8M	21.6M	77.3 ± 0.8	79.5
rank-128 factorised embedding	38.1M	21.6M	51.0 ± 5.0	57.6
Submission: masked KD, seed 21, vocab-pruned	**45.5M**	21.6M	**78.3** (1 ckpt)	82.7
same checkpoint before pruning	70.8M	21.6M	78.0 (1 ckpt)	82.7

Table 1: GLUE RTE dev accuracy (%). "Val" (mean best validation accuracy) is the only number used for selection. The submission is the highest-validation checkpoint of all 20 xsmall runs (4 methods × 5 seeds); dev played no part, and the 5-seed mean (78.3) is the fairer estimate of the method. (The best plain checkpoint, val 81.0, happens to score 80.9 on dev; picking it would have meant selecting on dev.)

both dev (+0.6) and validation (+0.7), but neither gain is significant over 5 seeds (p=0.57, 0.21). α=0.8, T=4 has the best mean validation (79.5) and the lowest seed spread (±0.8) but not better dev.

3 Accuracy vs. parameters

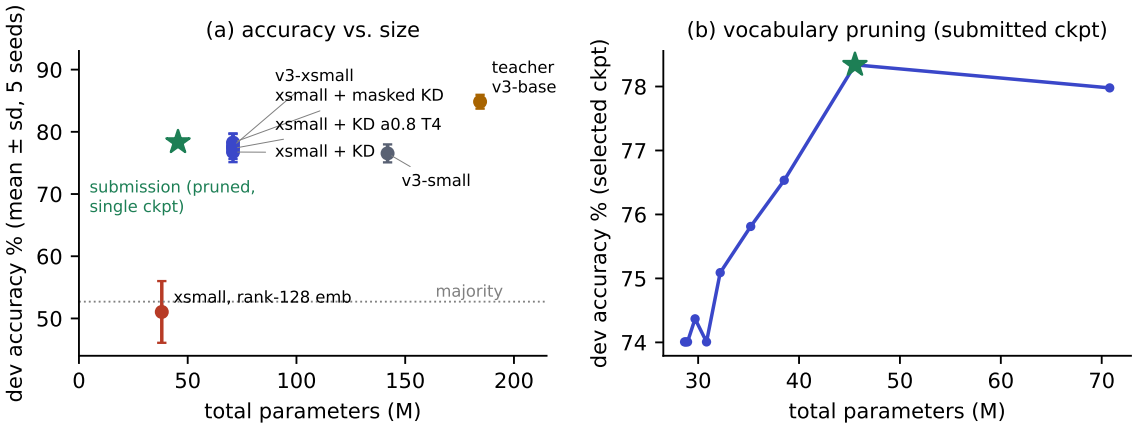


Figure 1: (a) Dev accuracy (mean ± sd over 5 seeds) against total parameters; the star is the pruned submission (single checkpoint). (b) Pruning curve for the submitted checkpoint: train tokens plus the top-N frequent tokens, $N \in \{0, \ldots, 60\text{k}, \text{all}\}$.

Larger is not automatically better here (Fig. 1a). v3-small has twice xsmall's parameters and twice its non-embedding parameters, but scored lower on both validation (75.1 vs. 78.5) and dev (76.5 vs. 77.7). The large jump is to v3-base (+7 points), which is wider (768 vs. 384 hidden) *and* deeper than small, so on this data the benefit comes from pretraining capacity per layer rather than raw parameter count.

Where parameters can be cut matters more than how many. Shrinking the embedding by SVD at rank 128 kept only 57.5% of its energy; training loss stayed at chance (0.70) for 4–5 epochs on every seed and the model never recovered (51.0%). Pruning the vocabulary instead removes rows that never fire, so the remaining model is unchanged: −36% parameters at no measurable cost (+0.4 points here, −0.7 on the plain seed-21 checkpoint). Fig. 1b shows a smaller version would do worse: keeping only training tokens (28.7M) costs 4.0 points, because unseen dev tokens are mostly rare names, and only 98.5% of dev tokens (60% of pairs) are fully covered even at N=60k.

4 Error analysis

The submission makes 60 errors on dev. 34 are false *entailment* predictions (error rate 26.0% on gold not-entailment vs. 17.8% on entailment); it still predicts entailment slightly too often (55.6% vs. 52.7% gold). Automatic tags, error rate on each subset (overall 21.7%): high lexical overlap (≥80%) but not entailed 36.8% (n=38); low overlap (<50%) but entailed 42.9% (14); numbers/dates in the hypothesis 31.2% (48); negation in exactly one side 22.5% (40); top-10% long premises 21.4% (28). Negation and length are *not* major error sources here.

I hand-labelled a random sample of 25 errors (seed 0; `report/error_annotation.csv`):

- **Overlap, unsupported claim (7, all false entailments).** The hypothesis reuses premise words but changes a role or adds a detail: Mr Dean released a plan "on behalf of the Secretary" → "Mr Dean is the Secretary"; "a substance found in red wine" → "red wine"; an added "60 million barrels". This is the lexical-overlap heuristic known from NLI models [6].
- **Paraphrase missed (5, false non-entailments).** "will spread out over a year" → "lasts a year"; "BSE, the so-called mad cow disease"; an appositive inside a long premise.
- **Numbers (5).** Binding: 80% approval belongs to Mrs. not Mr. Bush; 32 *populations* read as 32 pandas. Comparison: €72.090.405,19 > "more than 72 million"; 12,000 years ago ⇒ humans existed 10,000 years ago.
- **Modality and world knowledge (6).** "nearly 4 percent" → "4 percent"; "attempt" Everest → "succeeds"; asked to form a government ⇒ prime minister; once the model used outside knowledge the premise does not state (Bush's wife is Laura).
- **Other (2):** one near the threshold (p=0.39), one doubtful gold label (a state court read as the U.S. Supreme Court).

5 Reflection

Counting *where* the parameters are paid off most: post-hoc vocabulary pruning is cheap, exact on covered text, and gave the largest efficiency gain. Logit distillation helped only once the teacher's wrong labels were masked, and then within seed noise; factorising the embedding and a bigger student did not help. **With more time/compute:** (i) prune the vocabulary *before* fine-tuning and use the saved budget for a wider backbone; (ii) intermediate-layer or attention distillation instead of logits only, with the teacher's entailment bias corrected (class re-weighting or temperature per class); (iii) train-only data augmentation targeting the error classes above (role swaps, modality and quantifier edits of training hypotheses as not-entailment); (iv) more seeds to tighten the ±2-point seed spread, larger than most effects here.

Reproducibility. Pinned `uv` env; each run stores config, versions, parameter counts, per-seed results; Colab reruns are bit-identical. `make train-submission submission` rebuilds it.

References

[1] A. Abdaoui, C. Pradel, G. Sigel. Load What You Need: Smaller Versions of Multilingual BERT. *SustaiNLP*, 2020.

[2] P. He, X. Liu, J. Gao, W. Chen. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *ICLR*, 2021.

[3] P. He, J. Gao, W. Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. *ICLR*, 2023.

[4] G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*, 2015.

[5] T. Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. *ACL*, 2018.

[6] R. T. McCoy, E. Pavlick, T. Linzen. Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference. *ACL*, 2019.

[7] M. Mosbach, M. Andriushchenko, D. Klakow. On the Stability of Fine-tuning BERT. *ICLR*, 2021.